

RTL PROJECT

Prepared By

Rawan Al - oyayri
Renad Al - oyayri
Roaa ALnawshan

123-456-7890
@reallygreatsite
www.reallygreatsite.com

1. ALU INTRODUCT

Arithmetic Logic Unit (ALU) is one of the main components of the processor. It performs ,arithmetic operations on binary data. In this project a 4-bit ALU was designed to perform addition and .subtraction operations

2. ALU DESIGN

Inputs

signal	size	description
A	4-bit	first operand
B	4-bit	second operand
S	1-bit	operation select

outputs

signal	size	description
out	4-bit	result
carry	1-bit	carry out

OPERATION

$S = 0 \rightarrow \text{Addition (A + B)}$

$S = 1 \rightarrow \text{Subtraction (A - B)}$

:Subtraction is implemented using the 2's complement method

$A - B = A + (\text{2's complement of B})$

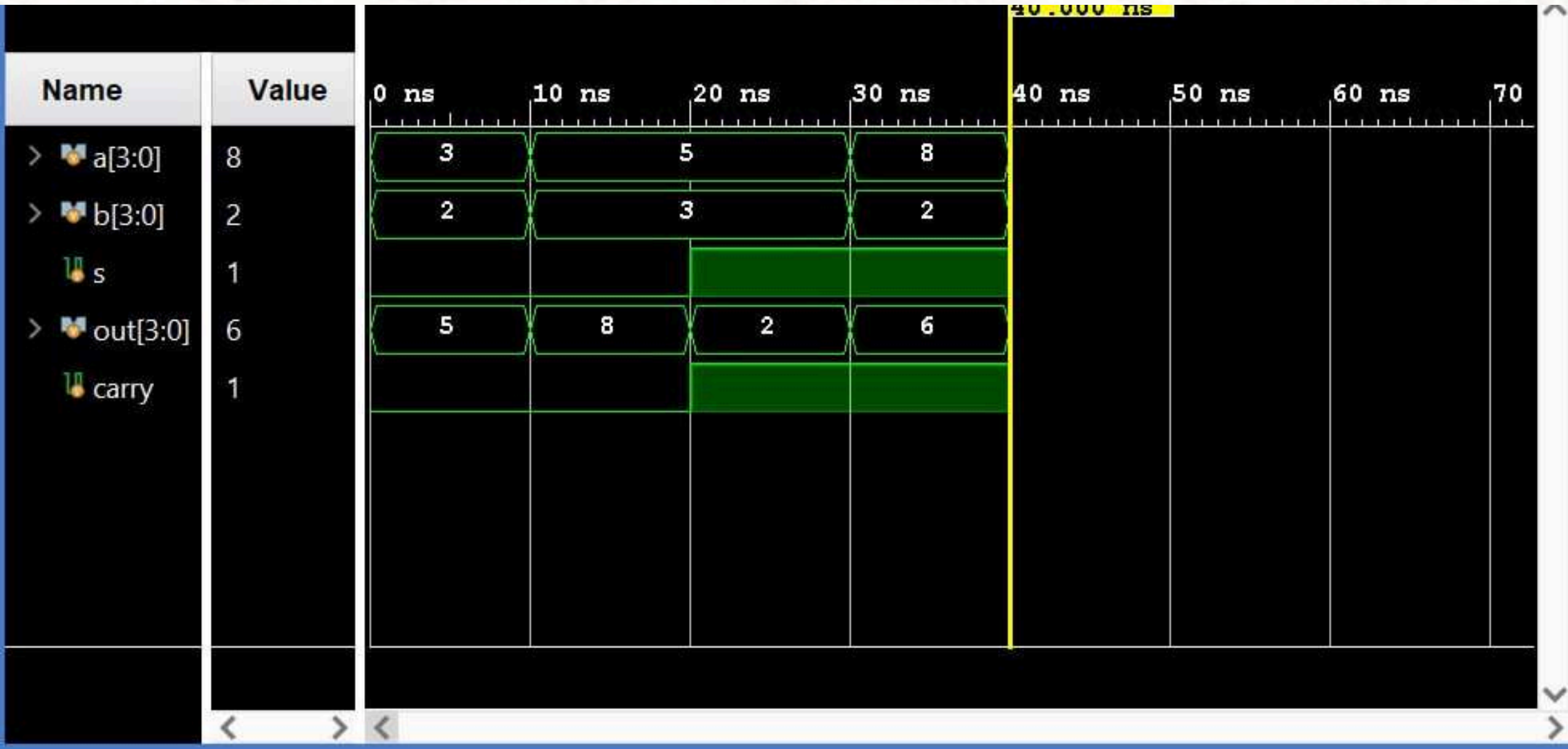
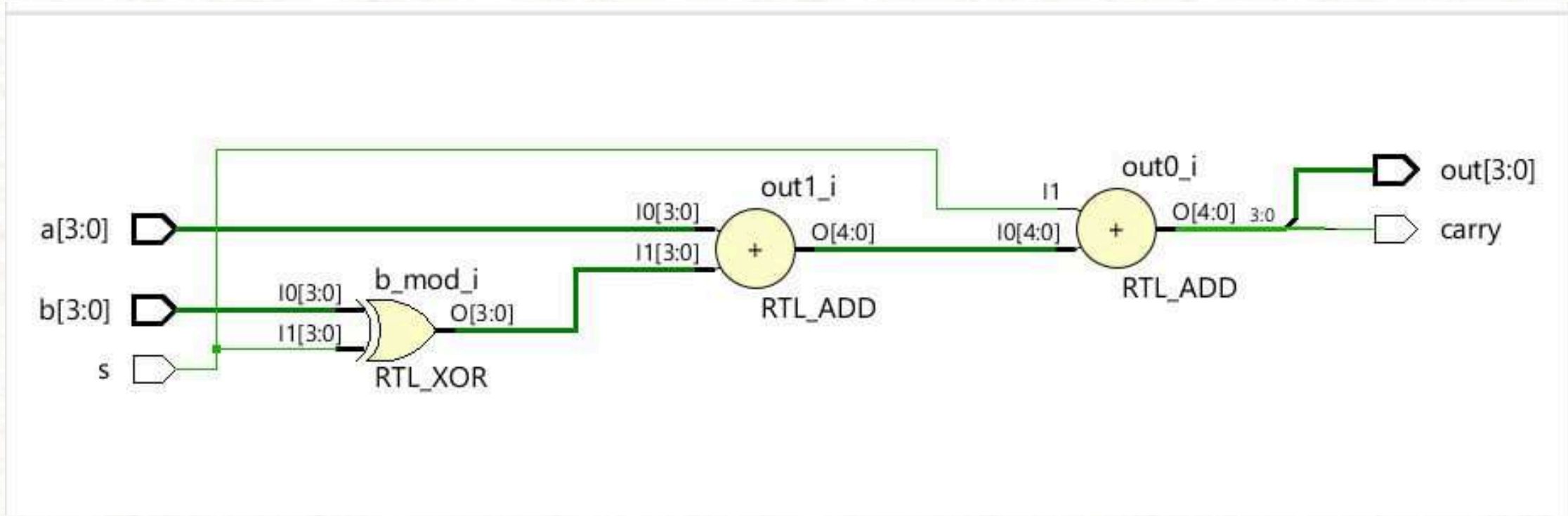
The bits of B are inverted using XOR gates and then 1 is added through the control signal S

3.SYSTEMVERILOG IMPLEMENTATION

```
module ALU(  
input logic[3:0] a, b ,  
input logic s,  
output logic [3:0]out,  
output logic carry  
);  
  
    logic [3:0]b_mod ;  
    logic [4:0]result ;  
  
    assign b_mod = b ^  
{4{s}};  
  
    always_comb begin  
        result= a + b_mod + s;  
        out= result[3:0];  
        carry=result[4];  
  
    end  
endmodule
```

The ALU was implemented using SystemVerilog. A modified version of input B was generated using XOR gates. The ALU performs addition when $S=0$ and subtraction when $S=1$. The carry output is extracted from the most significant bit of the result

4. SIMULATION RESULTS



The ALU was tested using a SystemVerilog testbench. Different test cases were applied for both addition and subtraction operations. The simulation results confirmed the correct functionality of the ALU

5: DATA REGISTERS

At least two 4-bit registers, **RA** and **RB**, are required to store the data used by the ALU. The outputs of these registers are directly connected to the two ALU inputs, A and B, respectively.

Therefore, the values stored in RA and RB are continuously available to the ALU.

The registers have controlled inputs, allowing new data to be loaded only when the corresponding enable signal is set to logic 1. When the enable signal is 0, the register retains its previous value.

This controlled storage allows the datapath to determine when new values are loaded into RA and RB during instruction execution.


```
module register
#(parameter width = 4
)
input logic clk, en,rstn,
input logic [width-1:0] d,
output logic [width-1:0] q
);

always_ff @(posedge clk or
negedge rstn ) begin

if (!rstn )
q<= '0;

else if(en)
q<=d;
end

endmodule
```



```
module datapath #(parameter width=4)(  
  input logic [width-1:0] imm,  
  input logic enA, enB, s, clk, rstn,  
  output logic carry,  
  output logic [width-1:0] out  
  );
```

```
  logic [width-1:0] RA_out;  
  logic [width-1:0] RB_out;
```

```
  register #(.width(width) ) RA(  
    .clk(clk),  
    .en(enA),  
    .rstn(rstn),  
    .d( imm),  
    .q(RA_out)  
  );
```

```
  register #(.width(width) ) RB(  
    .clk(clk),  
    .en(enB),  
    .rstn(rstn),  
    .d( imm),  
    .q(RB_out)  
  );
```

```
  ALU alu1(  
    .a(RA_out),  
    .b(RB_out),  
    .s(s),  
    .carry(carry),  
    .out(out)  
  );
```

```
endmodule
```



```
`timescale 1ns / 1ps
module datapath_tb;
parameter width=4;
logic [width-1:0] imm;
logic enA, enB, s, clk , rstn ;
logic carry;
logic [width-1:0] out;

datapath #(.width(width)) dut(
    .clk(clk),
    .enA(enA),
    .enB(enB),
    .rstn(rstn),
    .imm(imm),
    .s(s),
    .out(out),
    .carry(carry)
);

initial begin
    clk=0;
    forever
        #5
        clk=~clk;
    end
```



```
    initial begin  
rstn=0;  
enA=0;  
enB=0;  
imm=0;  
s=0;
```

```
    #10;  
rstn=1;
```

```
imm=4'b0101;  
enA=1;  
#10;  
enA=0;
```

```
imm=4'b1111;  
enB=1;  
#10;  
enB=0;
```

```
imm=4'b1001;  
enA=1;  
#10;  
enA=0;
```

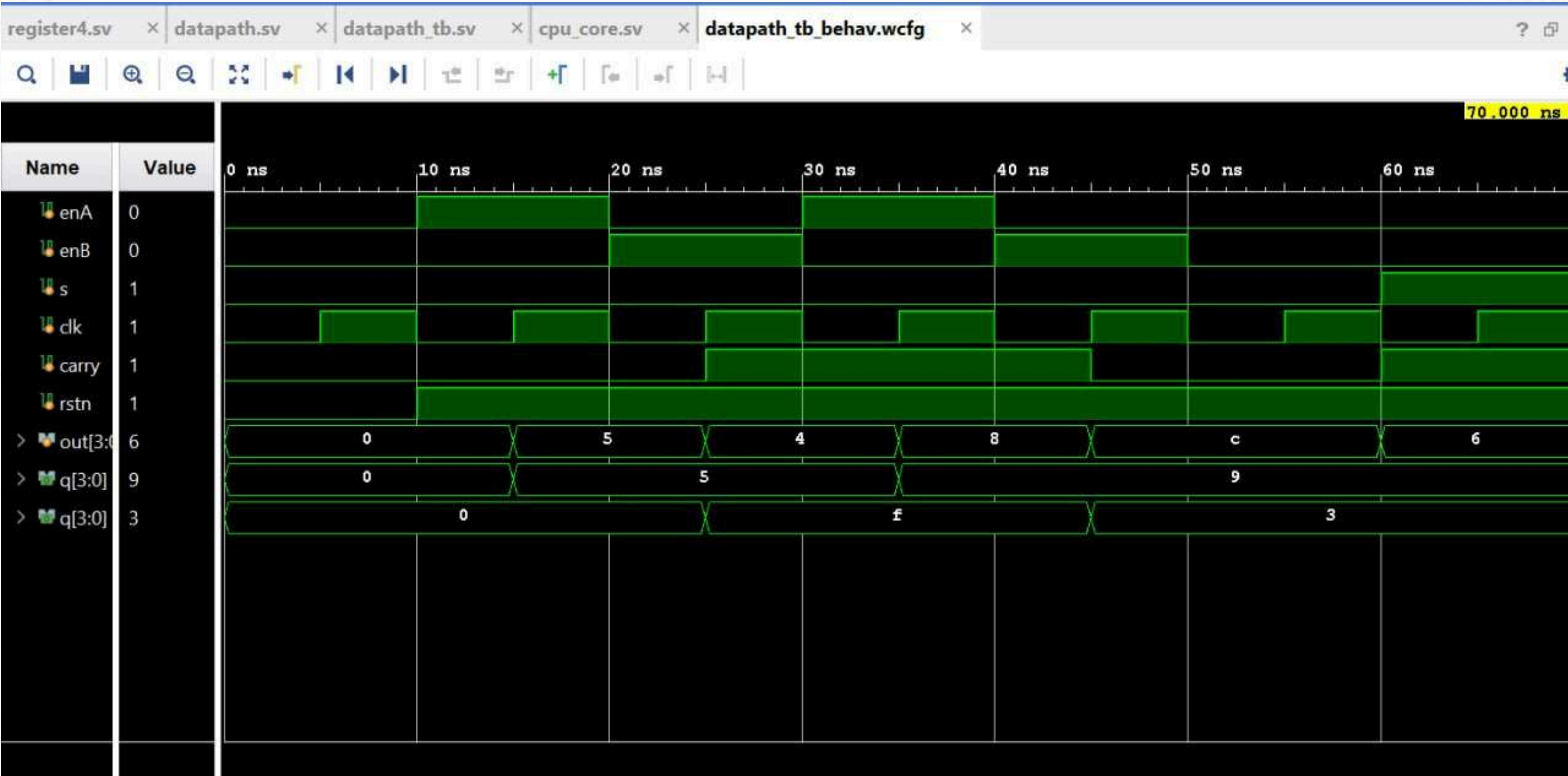
```
imm=4'b0011;  
enB=1;  
#10;  
enB=0;  
//add  
s=0; #10;
```

```
//sub  
s=1; #10;
```

```
$finish;
```

```
end  
endmodule
```


Testbench Simulation Results



5.1 Functionality and Time Synchronization

Objective

The objective of this part was to update the datapath so that the registers can either load an external value or store the ALU result.

Design

The following components were added:

- A **4-bit 2×1 Multiplexer** before each register.
- A control signal **Sreg** to select the MUX input.
- A **D Flip-Flop** to store the carry output from the ALU.

1 → External input (data_in)

0 → ALU output

The carry output of the ALU is stored in a D Flip-Flop on every positive clock edge.

Results

The updated datapath was simulated successfully.

The waveform verified that:

- Registers correctly load immediate when **Sreg = 1**.
- Registers correctly store the ALU output when **Sreg = 0**.
- The carry output is synchronized with the clock using the D Flip-Flop.


```
module mux2x1_4bit(  
    input logic [3:0] a,  
    input logic [3:0] b,  
    input logic sel,  
    output logic [3:0] y  
);
```

```
always_comb begin  
    if (sel)  
        y = b;  
    else  
        y = a;  
end
```

```
endmodule
```

```
module dff(  
    input logic clk,rstn,  
    input logic d,  
    output logic q  
);
```

```
always_ff @(posedge clk or negedge rstn) begin  
    if (!rstn )  
        q<= 1'b0;  
    else  
        q <= d;  
end
```

```
endmodule
```



```
module register # (parameter width=4)(  
    input logic clk, en,rstn,  
    input logic [width-1:0] d,  
    output logic [width-1:0] q  
);  
always_ff @(posedge clk or negedge rstn) begin  
    if(!rstn)  
        q<='0;  
    else if(en)  
        q<=d;  
end  
  
endmodule
```



```
module datapath #(
  parameter width= 4
)(
  input logic [width-1:0] imm,
  input logic clk,
  input logic rstn,
  input logic enA,
  input logic enB,
  input logic s,
  input logic Sreg,

  output logic [width-1:0] out,
  output logic carry
);

// Internal signals
logic [width-1:0] RA_out;
logic [width-1:0] RB_out;

logic [width-1:0] MUX_output;

logic carry_ALU;

mux2x1_4bit mux(
  .a(out),
  .b(imm),
  .sel(Sreg),
  .y(MUX_output)
);
```



```
// Register A
register #(
    .width(width)
) RA (
    .clk(clk),
    .en(enA),
    .rstn(rstn),
    .d(MUX_output),
    .q(RA_out)
);
```

```
// Register B
register #(
    .width(width)
) RB (
    .clk(clk),
    .en(enB),
    .rstn(rstn),
    .d(MUX_output),
    .q(RB_out)
);
```

```
// ALU
ALU alu1(
    .a(RA_out),
    .b(RB_out),
    .s(s),
    .out(out),
    .carry(carry_ALU)
);
```

```
// Carry Register
dff carry_ff(
    .clk(clk),
    .rstn(rstn),
    .d(carry_ALU),
    .q(carry)
);
endmodule
```



```

`timescale 1ns / 1ps
module datapath_tb;
  parameter width = 4;

  logic [width-1:0] imm;
  logic enA, enB, s, clk;
  logic rstn;
  logic Sreg;
  logic carry;
  logic [width-1:0] out;

  datapath #( .width(width) ) dut (
    .clk(clk),
    .rstn(rstn),
    .imm(imm),
    .enA(enA),
    .enB(enB),
    .s(s),
    .Sreg(Sreg),
    .out(out),
    .carry(carry)
  );

  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  initial begin

    rstn = 0;
    enA = 0;
    enB = 0;
    s = 0;
    Sreg = 1;    // Select 'imm' as input

```



```
#10;  
rstn = 1;  
#5;
```

```
imm = 4'b0101;  
enA = 1;  
#10;  
enA = 0;
```

```
imm = 4'b1111;  
enB = 1;  
#10;  
enB = 0;
```

```
imm = 4'b1001;  
enA = 1;  
#10;  
enA = 0;
```

```
imm = 4'b0011;  
enB = 1;  
#10;  
enB = 0;
```

```
s = 0;  
#10;
```

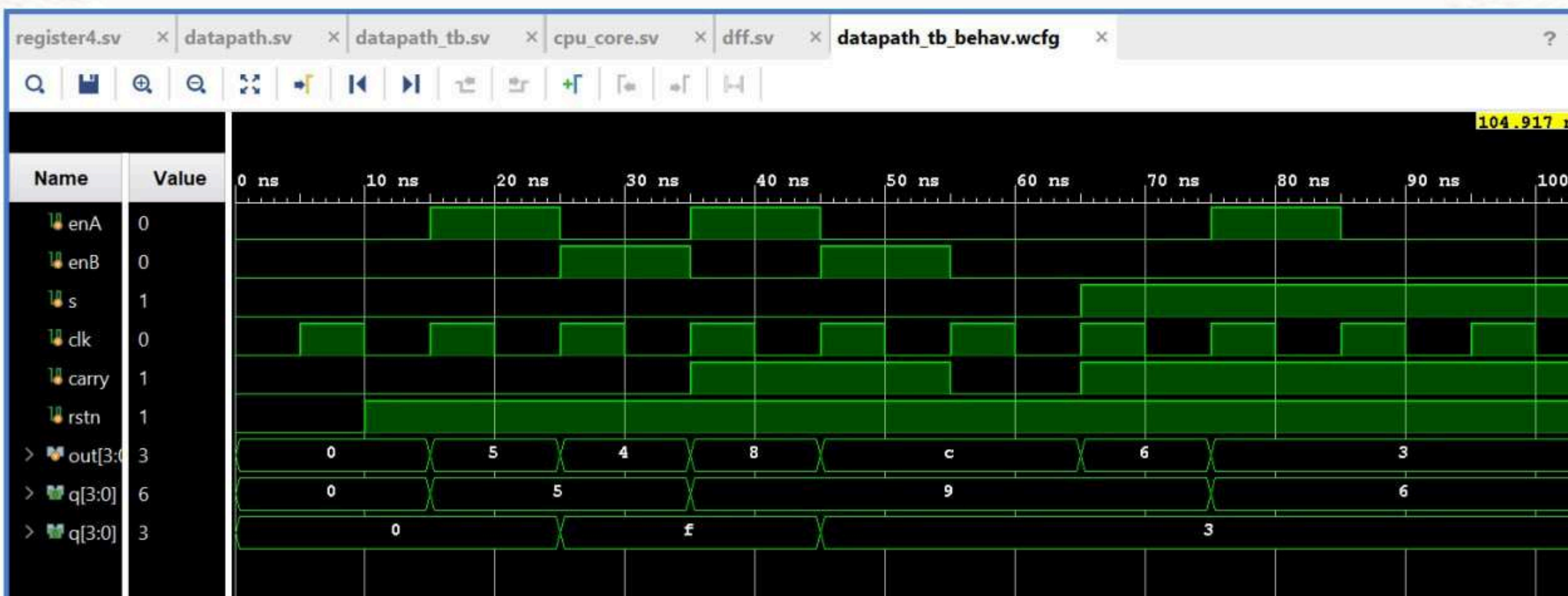
```
// SUB operation (s = 1)  
s = 1;  
#10;
```

```
Sreg = 0;  
enA = 1;  
#10;  
enA = 0;
```

```
#20;  
$finish;  
end
```

```
endmodule
```


Testbench Simulation Results



5.2 Testing

Objective

The datapath was tested by implementing a register swapping algorithm without using a temporary register.

Initial values:

$$\mathbf{RA = 2}$$

$$\mathbf{RB = 3}$$

The following sequence of operations was executed:

$$\mathbf{RA = RA + RB}$$

$$\mathbf{RB = RA - RB}$$

$$\mathbf{RA = RA - RB}$$

Expected Result

After executing the three operations:

$$\mathbf{RA = 3}$$

$$\mathbf{RB = 2}$$

Thus, the original values of RA and RB are successfully exchanged without requiring an additional temporary register.

Results

The simulation waveform confirmed that the three operations were executed correctly and that the values stored in RA and RB were successfully swapped.

This test demonstrates that the datapath can correctly store and reuse ALU results across multiple clock cycles.


```
`timescale 1ns / 1ps

module datapath_tb;

parameter width= 4;

logic [width-1:0] imm;
logic enA, enB, s, clk;
logic rstn;
logic Sreg;
logic carry;
logic [width-1:0] out;

datapath #(
    .width(width)
) dut (
    .clk(clk),
    .rstn(rstn),
    .imm(imm),
    .enA(enA),
    .enB(enB),
    .s(s),
    .Sreg(Sreg),
    .out(out),
    .carry(carry)
);

// Clock Generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end;

initial begin
    rstn = 0;
    enA = 0;
    enB = 0;
    Sreg = 1; // Load from imm
    s = 0;
    imm = 0;

    #10;
    rstn = 1;
```



```
// Load RA = 2
imm = 4'd2;
enA = 1;
#10;
enA = 0;

// Load RB = 3
imm = 4'd3;
enB = 1;
#10;
enB = 0;
// Swapping

// RA = RA + RB
Sreg = 0; // Take ALU output
s = 0; // Addition
enA = 1;
#10;
enA = 0;

// RB = RA - RB
s = 1; // Subtraction
enB = 1;
#10;
enB = 0;

// RA = RA - RB
s = 1;
enA = 1;
#10;
enA = 0;

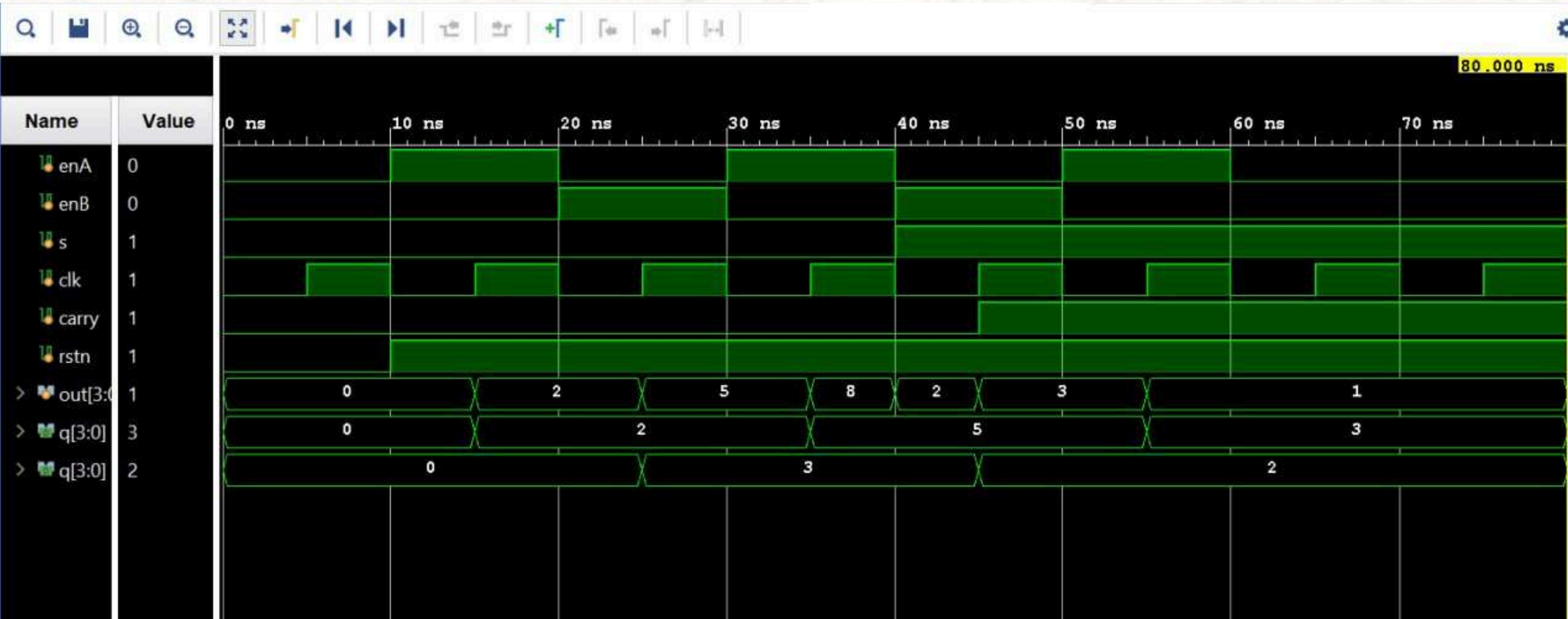
#20;

$finish;

end

endmodule
```


Register Swapping Simulation Waveform



5.3 Output Register

Objective

An output register, **Ro**, was added to store the final result produced by the datapath.

Design

A **4-bit output register Ro** was connected to the output of RA. An enable signal, **enOut**, controls when the output register captures the value from RA.

The register operates synchronously with the clock:

- **enOut = 1**: Ro loads the current value of RA on the positive clock edge.
- **enOut = 0**: Ro retains its previous value.

Results

Simulation verified that the output register correctly captures the value of RA when **enOut = 1**. When the enable signal is inactive, the output register maintains its previously stored value.


```
module datapath #(
    parameter width = 4
)(
    input logic [width-1:0] imm,clk,
    rstn,enA,enB,enO,s,Sreg,
    output logic [width-1:0] result,
    output logic [width-1:0] out,
    output logic carry
);
```

```
logic [width-1:0] RA_out;
logic [width-1:0] RB_out;
logic [width-1:0] MUX_output;
logic carry_ALU;
```

```
mux2x1_4bit mux(
    .a(result),
    .b(imm),
    .sel(Sreg),
    .y(MUX_output)
);
```

```
// Register A
register #(
    .width(width)
) RA (
    .clk(clk),
    .rstn(rstn),
    .en(enA),
    .d(MUX_output),
    .q(RA_out)
);
```

```
// Register B
register #(
    .width(width)
) RB (
    .clk(clk),
    .rstn(rstn),
    .en(enB),
    .d(MUX_output),
    .q(RB_out)
);
```



```
register #(
    .width(width)
) RO (
    .clk(clk),
    .rstn(rstn),
    .en(enO),
    .d(RA_out),
    .q(out)
);
```

```
ALU alu1(
    .a(RA_out),
    .b(RB_out),
    .s(s),
    .out(result),
    .carry(carry_ALU)
);
```

```
dff carry_ff(
    .clk(clk),
    .d(carry_ALU),
    .q(carry)
);
```

```
endmodule
```



```
`timescale 1ns / 1ps
module datapath_tb;
parameter width = 4;

logic [width-1:0] imm;
logic clk;
logic rstn;
logic enA;
logic enB;
logic enO;
logic s;
logic Sreg;
logic [width-1:0] out;
logic [width-1:0] result;

logic carry;
```

```
datapath #(
    .width(width)
) dut (
    .clk(clk),
    .rstn(rstn),
    .imm(imm),
    .enA(enA),
    .enB(enB),
    .enO(enO),
    .s(s),
    .Sreg(Sreg),
    .result(result),
    .out(out),
    .carry(carry)
);
```

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```


initial begin

```
rstn = 0;  
enA = 0;  
enB = 0;  
enO = 0;  
Sreg = 1;  
s = 0;  
imm = 0;
```

```
#10;
```

```
rstn = 1;
```

```
// Load RA = 2  
imm = 4'd2;  
enA = 1;
```

```
#10;
```

```
enA = 0;  
// Load RB = 3  
imm = 4'd3;  
enB = 1;
```

```
#10;
```

```
enB = 0;  
// RA = RA + RB  
// 2 + 3 = 5  
Sreg = 0;  
s = 0;
```

```
enA = 1;
```

```
#10;
```



```
enA = 0;  
// RB = RA - RB  
// 5 - 3 = 2  
s = 1;  
enB = 1;  
#10;
```

```
enB = 0;  
// RA = RA - RB  
// 5 - 2 = 3  
s = 1;
```

```
enA = 1;
```

```
#10;
```

```
enA = 0;  
// RO = RA  
// RO = 3
```

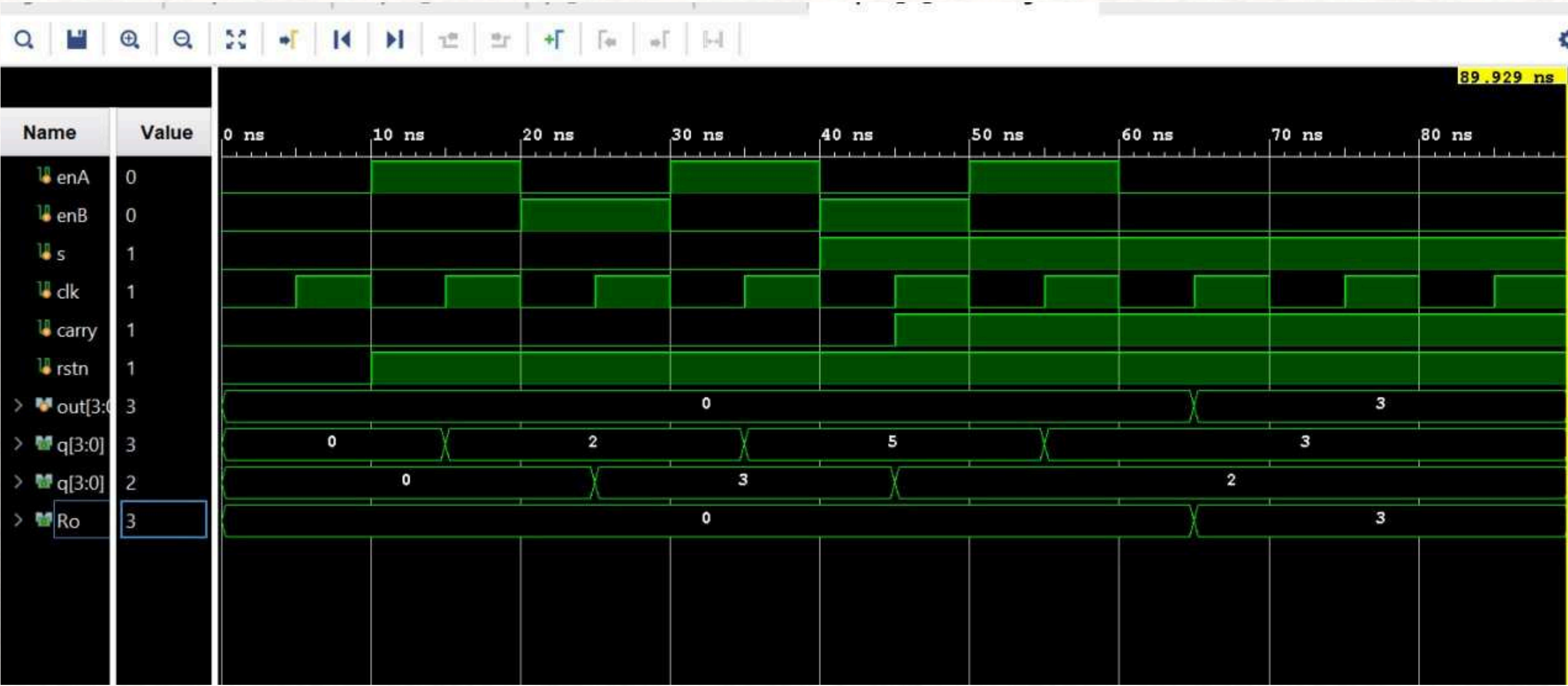
```
enO = 1;  
#10;  
enO = 0;  
#20;
```

```
$finish;
```

```
end
```

```
endmodule
```


Output Register Simulation Waveform



5.4 Output Enable

Objective

The purpose of this test was to verify the operation of the output enable signal (enOut) and ensure that the final datapath result is stored only when required.

Testing

Different values of enOut were applied during simulation to observe the behavior of the output register.

Results

The waveform verified the following behavior:

- **When enOut = 0**, Ro retains its previous value.
- **When enOut = 1**, Ro stores the current value of RA on the positive clock edge.
- The stored output remains stable until the register is enabled again.

Therefore, the output register successfully provides controlled storage of the final datapath result.

123-456-7890

@reallygreatsite

www.reallygreatsite.com


```
module decoder2x4(  
  
    input logic [1:0] w,  
  
    output logic y0,  
    output logic y1,  
    output logic y2,  
    output logic y3  
  
);  
  
always_comb begin  
    y0 = 0;  
    y1 = 0;  
    y2 = 0;  
    y3 = 0;  
    case (w)  
  
        2'b00: y0 = 1;  
        2'b01: y1 = 1;  
        2'b10: y2 = 1;  
        2'b11: y3 = 1;  
  
    endcase  
  
end  
  
endmodule
```



```
module datapath #(
    parameter width = 4
)(
    input logic clk,
    input logic rstn,
    input logic [width-1:0] imm,
    input logic s,
    input logic Sreg,
    input logic [1:0] w,
    output logic [width-1:0] result,
    output logic [width-1:0] out,
    output logic carry
);
logic [width-1:0] RA_out;
logic [width-1:0] RB_out;
logic [width-1:0] MUX_output;

logic carry_ALU;

logic enA;
logic enB;
logic enO;

mux2x1_4bit mux(
    .a(result),
    .b(imm),
    .sel(Sreg),
    .y(MUX_output)
);
register #(
    .width(width)
) RA(
    .clk(clk),
    .rstn(rstn),
    .en(enA),
    .d(MUX_output),
    .q(RA_out)
);
```



```
register #(
    .width(width)
) RB(
    .clk(clk),
    .rstn(rstn),
    .en(enB),
    .d(MUX_output),
    .q(RB_out)
);
```

```
register #(
    .width(width)
) RO(
    .clk(clk),
    .rstn(rstn),
    .en(enO),
    .d(RA_out),
    .q(out)
);
```

```
ALU alu1(
    .a(RA_out),
    .b(RB_out),
    .s(s),
    .out(result),
    .carry(carry_ALU)
);
```

```
dff carry_ff(
    .clk(clk),
    .rstn(rstn),
    .d(carry_ALU),
    .q(carry)
);
```

```
decoder2x4 dec1(
    .w(w),
    .y0(enA),
    .y1(enB),
    .y2(enO),
    .y3()
);
```

```
endmodule
```



```
`timescale 1ns / 1ps
module datapath_tb;
parameter width = 4;
logic clk;
logic rstn;
logic s;
logic Sreg;
logic [width-1:0] imm;
logic [1:0] w;
logic [width-1:0] out;
logic [width-1:0] result;
logic carry;
```

```
datapath #(
    .width(width)
) dut(
    .clk(clk),
    .rstn(rstn),
    .imm(imm),
    .Sreg(Sreg),
    .s(s),
    .w(w),
    .out(out),
    .carry(carry),
    .result(result)
);
initial begin
    clk = 0;
```

```
    forever #5 clk = ~clk;
end
```



```
initial begin
    rstn = 0;
    w = 2'b11;
    imm = 0;
    Sreg = 1;
    s = 0;
    #10;
    rstn = 1;

    // Load RA = 2

    imm = 4'd2;
    w = 2'b00; #10;
    w = 2'b11;

    // Load RB = 3
    imm = 4'd3;
    w = 2'b01; #10;
    w = 2'b11;

    // RA = RA + RB  2 + 3 = 5
    Sreg = 0;
    s = 0;
    w = 2'b00; #10;
    w = 2'b11;

    // RB = RA - RB  5 - 3 = 2
    s = 1;
    w = 2'b01; #10;
    w = 2'b11;

    // RA = RA - RB  5 - 2 = 3
    w = 2'b00; #10;
    w = 2'b11;

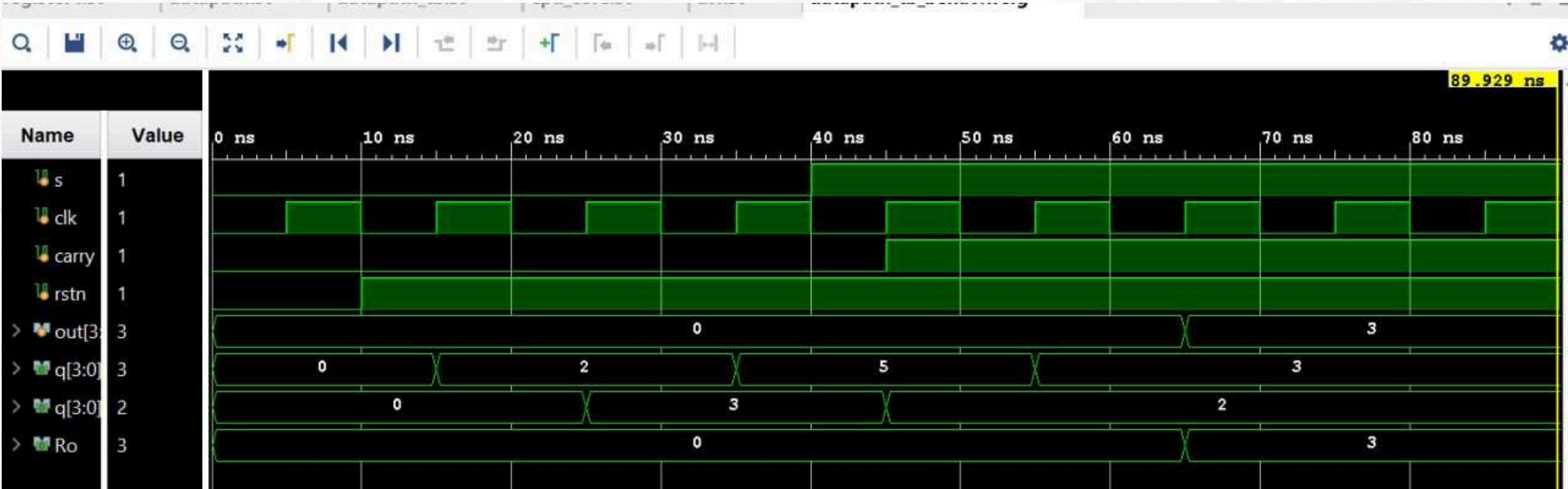
    // RO = RA
    w = 2'b10; #10;
    w = 2'b11; #20;

    $finish;

end

endmodule
```


Simulation Results



6. Control Logic

Logic values cannot be applied to the control signals manually at all times. To avoid this inconvenience, the values of the control signals are stored in a non-volatile memory (ROM) and connected to the design. The values of the control signals stored in the memory are called Instructions.

6.1 Instruction Encoding

Since the rows of instruction memory are 8 bits wide, all control signals are adjusted to fit within these 8 bits, with the custom input also forming part of the instruction:

- Jump (J): bit 7 (MSB). When set to 1, the custom input is loaded into the program counter.
- Carry jump (C): bit 6. When set to 1, the custom input is loaded into the program counter if the ALU produced a carry.
- Register Address (D1D0): bits 5–4. Values 00, 01, 10 and 11 correspond to RA, RB, RO and no register, respectively.
- ALU or Custom input (Sreg): bit 3. When 0, the ALU's output is directed to the register input; when 1, the custom input is used instead.
- Custom input (immediate): bits 2–0, giving values ranging from 0 to 7.
- ADD or SUB (S): bit 2 has a dual function — part of the custom input, and the ALU selection bit S — since these two uses never overlap in the same instruction.

The program counter's load condition follows directly from J and C:

Load = J OR (C AND Carry out)

Raising J to 1 makes the counter start counting from the custom input regardless of carry; raising C to 1 does the same only while the ALU carry-out is 1. This phenomenon is known as "JUMP" or "BRANCH" in computing language.

Results

The waveform verified the following behavior:

- **When enOut = 0**, Ro retains its previous value.
- **When enOut = 1**, Ro stores the current value of RA on the positive clock edge.
- The stored output remains stable until the register is enabled again.

Therefore, the output register successfully provides controlled storage of the final datapath result.

123-456-7890

@reallygreatsite

www.reallygreatsite.com

6.2 Control Unit

The Control Unit is one of the main components of the processor. It decodes the 8-bit instruction fetched from instruction memory and produces the individual control signals that drive every other block: the register-address lines, the ALU operation select, the register/immediate select, and the jump control bits. It was designed as a purely combinational module that maps each instruction bit directly to a named control signal.

INPUTS

signal	size	description
instr	8-bit	the fetched instruction

OUTPUT

signal	size	description
J	1-bit	unconditional jump flag
C	1-bit	jump-if-carry flag
addr	2-bit	register address (RA/RB/RO/none)
sreg	1-bit	ALU result vs immediate select
S	1-bit	ALU add/subtract select
imm	4-bit	immediate / jump-target value

The instruction format follows the specification exactly: bit 7 is J, bit 6 is C, bits 5-4 are the register address (D1D0), bit 3 is Sreg, and bit 2 doubles as the ALU select bit S when Sreg is 0, or as the top bit of the 3-bit immediate value when Sreg is 1. Bits 1-0 always carry the remaining immediate bits.

SystemVerilog Implementation

```
module control(  
    input logic [7:0] instr,  
    output logic      J,  
    output logic      C,  
    output logic [1:0] addr,  
    output logic      Sreg,  
    output logic      s,  
    output logic [3:0] imm  
);  
  
assign J  = instr[7];  
assign C  = instr[6];  
assign addr = instr[5:4];  
assign Sreg = instr[3];  
assign s   = instr[2];  
assign imm = {1'b0, instr[2:0]};  
  
endmodule
```

@reallygreatsite
www.reallygreatsite.com

The control unit was implemented using simple continuous assignments, since decoding an instruction requires no sequential logic — every output is a direct function of the current instruction bits.

Results

The control unit was verified in isolation against every function defined in the specification's instruction table: $RA=RA+RB$, $RB=RA+RB$, $RA=RA-RB$, $RB=RA-RB$, $RO=RA$, $RA=imm$, $RB=imm$, jump-if-carry, and unconditional jump. All nine test cases produced the expected J, C, addr, Sreg, s, and imm values, confirming the decoder matches the specification exactly.

6.3 Instruction Memory

The Instruction Memory stores the program that the processor executes. It is implemented as a small read-only memory (ROM) that returns an 8-bit instruction for every 4-bit address it receives. For Phase 1 testing, the memory was pre-loaded with the Fibonacci sequence program given in Section 7.1.

Input

signal	size	description
address	4-bit	instruction address from the Program Counter

Outputs

signal	size	description
instr	8-bit	the instruction stored at that address

SystemVerilog Implementation

```
module instruction_memory(  
    input logic [3:0] address,  
    output logic [7:0] instr  
);  
  
always_comb begin  
    case(address)  
        4'd0: instr = 8'b00001000; // RA = 0  
        4'd1: instr = 8'b00011001; // RB = 1  
        4'd2: instr = 8'b00100000; // RO = RA  
        4'd3: instr = 8'b00010000; // RB = RA + RB  
        4'd4: instr = 8'b01110000; // jump if carry  
        4'd5: instr = 8'b00000000; // RA = RA + RB  
        4'd6: instr = 8'b00010100; // RB = RA - RB  
        4'd7: instr = 8'b00000100; // RA = RA - RB  
        4'd8: instr = 8'b10110010; // jump to 2  
        default: instr = 8'b00000000;  
    endcase  
end  
  
endmodule
```


The memory is implemented combinatorially with a case statement, since instruction memory in this design is read-only and does not need to be clocked — the correct instruction appears as soon as the address changes.

Results

The instruction memory was verified by sweeping the address input from 0 to 8 and confirming that each output matched the Fibonacci program exactly, including the RA=0 and RB=1 initialization instructions, the RO=RA output instructions, the addition instruction, the carry-conditional jump back to the start, and the three-instruction register swap sequence.

6.4 Jump Logic

The Jump Logic determines when the Program Counter should load a new address instead of simply incrementing. Two jump conditions are supported, exactly as defined in the specification: an unconditional jump triggered by the J control bit, and a conditional jump that only triggers when the ALU has produced a carry out, controlled by the C bit.

Input

signal	size	description
J	1-bit	unconditional jump control bit
C	1-bit	jump-if-carry control bit
carry	1-bit	carry flag from the ALU

Output

signal	size	description
load	1-bit	asserted when the Program Counter must jump

SystemVerilog Implementation

```
module jump_logic(  
    input logic J,  
    input logic C,  
    input logic carry,  
    output logic load  
);  
  
assign load = J | (C & carry);  
  
endmodule
```

Results

The jump logic was tested for every combination of J, C, and carry. With J=0, C=0, load stayed low, allowing the Program Counter to simply increment. With J=1, load was asserted regardless of carry, confirming the unconditional jump. With J=0, C=1, load was only asserted when carry=1 and remained low when carry=0, confirming the conditional jump works only under the intended overflow condition.

6.5 Program Counter

The Program Counter tracks the address of the instruction currently being fetched from instruction memory. On every clock edge it either increments by one for normal sequential execution, or loads a new address supplied by the current instruction when a jump condition is active, implementing both the JUMP and BRANCH behaviour described in Section 6.1

Input

signal	size	description
clk	1-bit	system clock
rstn	1-bit	active-low synchronous reset
load	1-bit	asserted to jump instead of incrementing
imm	4-bit	jump target address

Output

signal	size	description
count	4-bit	current instruction address

SystemVerilog Implementation

```
module program_counter #(
    parameter width = 4
)(
    input logic clk, rstn, load,
    input logic [width-1:0] imm,
    output logic [width-1:0] count
);

always_ff @(posedge clk) begin
    if (!rstn)
        count <= '0;
    else if (load)
        count <= imm;
    else
        count <= count + 1'b1;
end

endmodule
```

The counter is parameterized by width so it can be reused for wider address spaces. On reset it clears to zero; otherwise it either loads the jump target or increments by one every clock edge.

Results

The program counter was verified in isolation: after reset the count correctly held at zero, it then incremented sequentially (0, 1, 2, 3...) while load was low, it jumped immediately to the supplied immediate value whenever load was asserted, and it resumed normal incrementing from the new address afterward.

6.6 CPU Integration (t2_processor)

After each block was verified individually, the Control Unit, Datapath, Instruction Memory, Jump Logic, and Program Counter were connected together into a single top-level module, t2_processor. This closes the complete fetch-decode-execute loop described in the project specification: the Program Counter addresses the Instruction Memory, the fetched instruction drives the Control Unit, the Control Unit's signals drive the Datapath, the Datapath's carry output feeds the Jump Logic together with J and C from the Control Unit, and the Jump Logic's load signal returns to the Program Counter to decide the next address.

Input

signal	size	description
clk	1-bit	system clock
rstn	1-bit	active-low synchronous reset

Outputs

signal	size	description
out	4-bit	output register (RO) value
result	4-bit	live ALU result
carry	1-bit	ALU carry flag

Only clk and rstn are external inputs; every other signal (instr, J, C, addr, Sreg, s, imm, load, and the Program Counter's address) is generated and consumed internally, so the processor runs the stored program entirely on its own once released from reset.

SystemVerilog Implementation

```
module t2_processor(  
    input logic    clk,  
    input logic    rstn,  
    output logic [3:0] out,  
    output logic [3:0] result,  
    output logic    carry  
);  
  
    logic [3:0] pc_count;  
    logic [7:0] instr;  
    logic J, C, load;  
    logic Sreg, s;  
    logic [1:0] addr;  
    logic [3:0] imm;  
  
    instruction_memory imem(  
        .address(pc_count),  
        .instr(instr)  
    );  
  
    control cu(  
        .instr(instr),  
        .J(J), .C(C),  
        .addr(addr), .Sreg(Sreg), .s(s), .imm(imm)  
    );
```

123-456-7890

@reallygreatsite

www.reallygreatsite.com


```
datapath dp(  
  .clk(clk),  
  .rstn(rstn),  
  .imm(imm),  
  .s(s),  
  .Sreg(Sreg),  
  .w(addr),  
  .out(out),  
  .carry(carry),  
  .result(result)  
);
```

```
jump_logic jl(  
  .clk(clk),  
  .rstn(rstn),  
  .imm(imm),  
  .s(s),  
  .Sreg(Sreg),  
  .w(addr),  
  .out(out),  
  .carry(carry),  
  .result(result)  
);
```


Results

The fully integrated processor was simulated with only clk and rstn driven externally. After releasing reset, the Program Counter began incrementing and fetching real instructions from memory, the Control Unit correctly decoded each one, and the Datapath executed them in sequence, confirming that the complete closed-loop path (PC → Instruction Memory → Control Unit → Datapath → Jump Logic → PC) operates correctly without any manual intervention

7. Requirements

There are 9 functions the processor can perform. The functions and the set of control signals for them are:

Function	J	C	D1	D0	Sreg	S	Custom Input
RA = RA + RB	0	0	0	0	0	0	000
RB = RA + RB	0	0	0	1	0	0	000
RA = RA - RB	0	0	0	0	0	1	000

7.1 Fibonacci Numbers (Phase 1 testing)

The following program produces the Fibonacci sequence in the output register (implemented in the instruction_memory module, Section 6.3):

#	Instruction	Binary	Comment
0	RA = 0	00001000	load zero to A
1	RB = 1	00011001	load 1 to B
2	RO = RA	00100000	push A to output
3	RB = RA + RB	00010000	add A to B
4	jump to 0 if carry	01110000	if A+B produces a carry, jump to start
5	RA = RA + RB	00000000	swap A and B (step 1)
6	RB = RA - RB	00010100	swap A and B (step 2)
7	RA = RA - RB	00000100	swap A and B (step 3)

Testbench

A dedicated testbench, tb_t2_processor, was written for the fully integrated processor. Unlike the individual unit testbenches, this testbench supplies only the clock and reset signals — the instruction stream is no longer provided manually, since it is generated internally by the Program Counter and Instruction Memory working together.

```
`timescale 1ns/1ps

module tb_t2_processor;

    logicclk, rstn;
    logic [3:0] out, result;
    logiccarry;

    t2_processor dut(
        .clk(clk),
        .rstn(rstn),
        .out(out),
        .result(result),
        .carry(carry)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rstn = 0;
        repeat (2) @(posedge clk);
        rstn = 1;

        repeat (200) @(posedge clk);
        $finish;
    end

endmodule
```


Reset is held low for two full clock cycles before being released, avoiding any race condition between the testbench and the Program Counter's reset logic. The simulation is then run for two hundred clock cycles, long enough to observe several complete passes through the Fibonacci program.

Fibonacci Test Objective

The goal of this test is to confirm that the processor executes the Fibonacci program exactly as intended: starting from RA=0 and RB=1, the sequence 0, 1, 1, 2, 3, 5, 8, ... should appear in the Output Register (RO) as the program loops, and the jump-if-carry instruction should return execution to the start of the loop whenever an addition overflows the 4-bit register, allowing the sequence to regenerate indefinitely.

Results

The waveform was inspected with the Program Counter's count, the Instruction Memory's instr output, and the processor's out signal added to the wave window. The Program Counter was confirmed to increment correctly through addresses 0 through 8 and jump back to address 2 whenever a carry was produced, matching the loop structure of the Fibonacci program. The Output Register captured each new Fibonacci value in turn, confirming that the fully integrated processor correctly executes a stored program from instruction memory with no manual instruction input, satisfying the Phase 1 requirement of the project.

7.2 Adding Support for Data Memory and Jump if Zero

Four new functions were added: RA = [imm] (load from data memory), RB = [imm] (load from data memory), [imm] = RA (store to data memory), and JZ imm (jump if the ALU result is zero). The requirement is that none of the original 9 instructions may be affected by this addition.

7.2.1 Instruction Encoding

Objective

The original 8-bit instruction format was fully committed, with every bit assigned a fixed meaning. To add four new functions without disturbing the existing nine, an unused bit combination needed to be identified.

Design

In the original table, J and C are never both 1 at the same time (J = 1 is used only for unconditional jump, C = 1 only for jump-if-carry). This combination (J = 1, C = 1) was therefore repurposed as a "memory-instruction prefix." When detected, the D1 D0 field selects one of four new operations instead of a register address:

Function	J	C	D1	D0	Sreg	S	Custom Input
RA = [IMM]	1	1	0	0	X	X	imm[2:0]
RB = [IMM]	1	1	0	1	X	X	imm[2:0]
[IMM] = RA	1	1	1	0	X	X	imm[2:0]
JZ IMM	1	1	1	1	X	X	imm[2:0]

Results

Because J = 1, C = 1 was previously unused, this scheme guarantees that none of the original nine instructions change behavior

7.2.2 Control Unit Extension

Objective

The control unit needed to detect the new memory prefix and generate the additional control signals required by the datapath, without altering the decoding of the original instructions.

SystemVerilog Implementation

```
module control(  
    input logic [7:0] instr,  
    output logic      J,  
    output logic      C,  
    output logic      Jz,  
    output logic [1:0] addr,  
    output logic      Sreg,  
    output logic      s,  
    output logic [3:0] imm,  
    output logic      memWrite,  
    output logic      memToReg  
);  
    logic mem_op;  
    assign mem_op = instr[7] & instr[6];  
  
    assign J= (~mem_op) & instr[7];  
    assign C= (~mem_op) & instr[6];  
    assign addr = instr[5:4];  
    assign Sreg = instr[3];  
    assign s= instr[2];  
    assign imm= {1'b0, instr[2:0]};  
  
    assign memToReg = mem_op & (instr[5:4] == 2'b00 | instr[5:4] ==  
2'b01);  
    assign memWrite = mem_op & (instr[5:4] == 2'b10);  
    assign Jz      = mem_op & (instr[5:4] == 2'b11);  
endmodule
```


Results

J and C are forced to 0 whenever the memory prefix is active, so a memory instruction can never trigger the original jump logic. Three new outputs — memToReg, memWrite and Jz — are derived from the D1 D0 field only when the prefix is present, leaving the original signal paths completely unchanged when mem_op = 0.

7.2.3 Data Memory

Objective

A small data memory was required to support loading a value into RA/RB from a given address and storing RA's value to a given address.

SystemVerilog Implementation

```
module data_memory #(
    parameter width = 4,
    parameter depth = 8
)(
    input logic      clk,
    input logic      we,
    input logic [2:0]  addr,
    input logic [width-1:0] din,
    output logic [width-1:0] dout
);
    logic [width-1:0] mem [0:depth-1];

    always_ff @(posedge clk) begin
        if (we)
            mem[addr] <= din;
    end

    assign dout = mem[addr];
endmodule
```

Results

The memory is 8 words × 4 bits, matching the 3-bit immediate address already used elsewhere in the design. Writes occur synchronously on the positive clock edge when we (memWrite) is high; reads are combinational, so the loaded value is available to the datapath in the same cycle the load instruction executes.

7.2.4 Datapath Extension

Objective

RA and RB needed a third possible data source (memory) in addition to the existing ALU-result and immediate-value paths, and the register-enable logic needed to avoid unintended side effects during a store.

Design

A second 2-to-1 multiplexer was added after the original Sreg-controlled mux, selecting between the existing (ALU/immediate) path and the new memory-read data, controlled by memToReg. A data_memory instance was added, written from RA_out and always read at address imm[2:0]. Because D1 D0 = 10 also corresponds to the original "RO = RA" enable code, the decoder's input is forced to the unused code 2'b11 during a store, preventing RO from being accidentally overwritten.

SystemVerilog Implementation

```
module datapath #(
    parameter width = 4
)(
    input logic      clk,
    input logic      rstn,
    input logic [width-1:0] imm,
    input logic      s,
    input logic      Sreg,
    input logic [1:0] w,
    input logic      memWrite,
    input logic      memToReg,
    output logic [width-1:0] result,
    output logic [width-1:0] out,
    output logic      carry,
    output logic      zero
);
    logic [width-1:0] RA_out, RB_out, MUX_output, mem_data, reg_in;
    logic      carry_ALU;
    logic      enA, enB, enO;
    logic [1:0] dec_in;

    mux2x1_4bit mux(
        .a(result), .b(imm), .sel(Sreg), .y(MUX_output)
    );

    mux2x1_4bit mem_mux(
        .a(MUX_output), .b(mem_data), .sel(memToReg), .y(reg_in)
    );

    register #(.width(width)) RA(
        .clk(clk), .rstn(rstn), .en(enA), .d(reg_in), .q(RA_out)
    );

    register #(.width(width)) RB(
        .clk(clk), .rstn(rstn), .en(enB), .d(reg_in), .q(RB_out)
    );
endmodule
```

@reallygreatsite
www.reallygreatsite.com


```
register #(.width(width)) RO(  
.clk(clk), .rstn(rstn), .en(enO), .d(RA_out), .q(out)  
);
```

```
ALU alu1(  
.a(RA_out), .b(RB_out), .s(s),  
.out(result), .carry(carry_ALU)  
);
```

```
dff carry_ff(  
.clk(clk), .rstn(rstn), .d(carry_ALU), .q(carry)  
);
```

```
assign zero = (result == {width{1'b0}});
```

```
data_memory #(.width(width)) dmem(  
.clk(clk), .we(memWrite), .addr(imm[2:0]),  
.din(RA_out), .dout(mem_data)  
);
```

```
assign dec_in = memWrite ? 2'b11 : w;
```

```
decoder2x4 dec1(  
.w(dec_in),  
.y0(enA), .y1(enB), .y2(enO), .y3()  
);
```

```
endmodule
```

123-456-7890

@reallygreatsite

www.reallygreatsite.com

Results

Simulation confirmed that RA/RB correctly load from memory when memToReg = 1, that a store writes RA's value to memory without disturbing RO or any register, and that a zero flag is asserted whenever the ALU result equals 0000.

7.2.5 Jump-if-Zero Logic

Objective

The program counter's load condition needed a third trigger — jumping when the ALU result is zero — in addition to the original unconditional and jump-if-carry conditions

SystemVerilog Implementation

```
module jump_logic(  
    input logic J,  
    input logic C,  
    input logic Jz,  
    input logic carry,  
    input logic zero,  
    output logic load  
);  
    assign load = J | (C & carry) | (Jz & zero);  
endmodule
```

Results

The original expression $J \mid (C \ \& \ \text{carry})$ is preserved unmodified as the first two terms of the OR, so the original jump and jump-if-carry instructions behave exactly as before. The new term $(Jz \ \& \ \text{zero})$ only activates the load signal when a JZ instruction is executing and the previous ALU result was zero.

7.2.6 Top-Level Integration

Objective

The top-level processor module needed to be rewired to pass the new memWrite, memToReg, Jz and zero signals between the control unit, datapath and jump logic.

SystemVerilog Implementation

```
module cpu_core(  
    input logic    clk,  
    input logic    rstn,  
    input logic [7:0] instr,  
    output logic [3:0] out,  
    output logic [3:0] result,  
    output logic    carry,  
    output logic    J,  
    output logic    C,  
    output logic    Jz,  
    output logic    zero  
);  
    logic Sreg, s, memWrite, memToReg;  
    logic [1:0] addr;  
    logic [3:0] imm;  
  
    control cu(  
        .instr(instr),  
        .J(J), .C(C), .Jz(Jz),  
        .addr(addr), .Sreg(Sreg), .s(s), .imm(imm),  
        .memWrite(memWrite), .memToReg(memToReg)  
    );  
  
    datapath dp(  
        .clk(clk), .rstn(rstn), .imm(imm), .s(s), .Sreg(Sreg),  
        .w(addr), .memWrite(memWrite),  
        .memToReg(memToReg),  
        .out(out), .carry(carry), .result(result), .zero(zero)  
    );  
endmodule
```

123-456-7890

@reallygreatsite

www.reallygreatsite.com

Results

The extended `cpu_core` exposes `Jz` and `zero` alongside the original `J`, `C` and carry outputs, so the surrounding `jump_logic` and `program_counter` modules can be connected without any change to their own interfaces beyond the two new inputs already described in Section 7.2.5.

8. Concluding Remarks

The design of a T2 processor presented in this document is actually a processor core. All the processing units integrated into laptops, mobiles and embedded devices are based on the same basic principle. Other interesting programs can be implemented, such as multiplying two numbers or generating multiplication tables. The processor's functionality can be extended by adding more operations to the ALU, or by increasing the data chunk size from 4 bits to 6, 8, or more bits. Successfully completing and extending this project, as done in Section 7.2, demonstrates a strong hold on the basics of processor design.


```
module program_counter #(
    parameter width = 4
)(
    input logic clk,rstn,load,
    input logic [width-1:0] imm,
    output logic [width-1:0] count
);

always_ff @(posedge clk) begin
    if (!rstn)
        count <= '0;
    else if (load)
        count <= imm;
    else
        count <= count + 1'b1;
end

endmodule
```